Project 1 Report: A Comparison of Solutions for Poisson's and Laplace's Equations

Dylan Lovato

March 2026

1 Introduction

It is often the circumstance that, for some physical problem, there is a corresponding partial differential equation (PDE) that requires a solution, but that the solution is difficult (or even impossible) to find through the usual analysis. A solution can still be approximated, however, using numerical methods.

Such a solution is found here for Poisson's equation and Laplace's equation, understanding that the latter is simply a special case of the former. Specifically, this is done for their 2-dimensional variations, and for certain boundary conditions and a particular source term in Poisson's equation. Note that the context considered here is electrostatics, but both equations may be applied in other circumstances

Poisson's equation in electrostatics is given by

$$\nabla^2 U = -\frac{\rho}{\epsilon_0}, \tag{1}$$

in which U is the electric potential and ρ is the charge density in some area, and ϵ_0 is simply the vacuum permittivity constant. The right-hand side of this equation is often called the 'source term', and is of course important to finding U. In the 2-dimensional case, this corresponds to

$$\frac{\partial^2 U}{\partial x^2} + \frac{\partial^2 U}{\partial y^2} = -\frac{\rho}{\epsilon_0}. \tag{2}$$

Solving for U analytically when the source term isn't simple can be very difficult; generally, a separation of variables is applied, but this doesn't always yield a solution. Regardless, this problem is a rather relevant one: given some area, subject to some constant potentials at the boundaries and some charge density, find the potential field which fills it. As such, it is prime example for a use case of numerical methods to solve PDEs.

Laplace's equation is simply a special case of this, in which we consider now the potential in an area with no charge whose boundaries are again held at a constant potential. This is given by

$$\frac{\partial^2 U}{\partial x^2} + \frac{\partial^2 U}{\partial y^2} = 0. \tag{3}$$

In this project, these PDEs are solved by two numerical methods, and through this these methods are assessed. The first is a traditional numerical method, Succesive Overrelaxation (SOR), and the second is a physics-informed neural network (PINN). In particular, the computational time required and the accuracy of the created solution are inspected and compared so as to determine which methodology may be more useful in general to solve similar problems.

Though Poisson's equation, involving some actual charge density, is solved, more focus is placed on Laplace's equation, as it has an analytical solution against which the numerical values may be compared.

2 Methods

First, Poisson's/Laplace's equation are solved using a traditional numerical method, SOR.

Poisson's equation may be approximated using finite differences, for a value U_{ij} on a discrete grid, as

$$\frac{U_{(i+1)j} - 2U_{ij} + U_{(i-1)j}}{\Delta x^2} + \frac{U_{i(j+1)} - 2U_{ij} + U_{i(j-1)}}{\Delta y^2}$$

$$= f(x, y)$$

for a horizontal space between discrete points of Δx, a vertical space of Δy, and $f(x, y) = -\frac{\rho}{\epsilon_0}$. Note that $\Delta x = \Delta y$ for a square grid of points. Solving for the potential at the point in question, U_{ij}, yields

$$U_{ij} = \frac{1}{4}(U_{(i+1)j} + U_{(i-1)j} + U_{i(j+1)} + U_{i(j-1)} - f(x,y)\Delta x^2). \tag{4}$$

Jacobi's method proceeds by iterating on this formula: beginning with some guess for the non-boundary values (here, the first guess is that all are zero), equation (4) is applied to every internal point, using the values found in the previous iteration for the right-hand side. A simple improvement to the algorithm comes with Gauss-Seidel, where the values used in the right-hand side of (4) are simply whatever the newest value for that particular point is (that is, if it has already been updated during the current iteration, that value is used instead for the calculation).

A more major improvement to the Jacobi method comes in the form of Succesive Overrelaxation, or SOR. The calculation is tweaked a little, as

$$U_{ij,m} = U_{ij,(m-1)} + \frac{\omega}{4}(U_{(i+1)j} + U_{(i-1)j} + U_{i(j+1)} + U_{i(j-1)} - 4U_{ij,(m-1)} - f(x,y)\Delta x^2) \tag{5}$$

for an iteration m (where m is not listed, whatever the newest value is used, m or $(m-1)$).

In this circumstance, the found Gauss-Seidel value is used as a sort of weighted correction to whatever the current value is, with a weight ω. This weight is between 1 and 2 (with $\omega = 1$ simply being the Gauss-Seidel expression), and may be determined heuristically. Here, though, there is a precise 'ideal' value for convergence of the algorithm, being $\omega = \frac{2}{1+sin(\frac{\pi}{N+1})}$ [1], for N^2 total discrete points, or a square with N points on each side.

Both Jacobi and SOR have the same converge criteria: the coefficient matrix created from equation (4) when applied to every point on the grid must be symmetric and positive-definite. The actual rate of convergence for SOR is about the square root of that for Jacobi, making it significantly more efficient. The number of iterations for SOR is theoretically $\frac{N+1}{2\pi}$. [1]

The grid used is 150 by 150, over 1 square meter ($N = 150$).

This methodology is tested against a physics-informed neural network, or PINN. The PINN used is a typical neural network trained to solve a PDE, and is specifically based on the PyTorch Python library, which is meant to facilitate precisely this.

The model is trained by optimizing a loss function. For some number of iterations, or epochs, the model is given a random sets of points along the boundaries of the area and in the interior. In each epoch, it produces a value based on its current parameters for each point. For boundary points, the value is already known, and so the loss is the difference between it and the value found by the PINN. For interior points, the right-hand side of the PDE is known; and so the PINN finds a value for U, and then automatic differentiation is used to find the left-hand side of the PDE. Loss here is thus the difference between these, or the residual. The L_2 error is then found, which is simply the sum of the squares of all of the losses (for all points in an epoch). This is passed to the optimizer, which uses automatic differentiation to alter the parameters in the PINN to, ideally, reduce the loss.

The parameters of the PINN correspond to coefficients in the layers; it takes in a point, passes it through three linear layers and two hyperbolic tangent layers, which then produce a potential based on their parameters. Training minimizes the loss, and as such should optimize the parameters to create a solution for the PDE for the particular context. This only needs to be done once;

values can be found at any point afterward by using the found parameters.

As a note; a learning rate of 0.001 is used.

3 Results

The SOR method found the solution for Poisson's equation, with all boundaries held at 0 Volts and a source term of

$$f(x,y) = -10exp(-100((x-0.5)^2+(y-0.5)^2)),$$

shown in FIG. 1. This result was achieved after 300 iterations, accomplished in approximately 15 seconds.

The PINN solution for Poisson's equation, in FIG. 2, was accomplished after training for 25000 epochs in around 5 minutes, ending with a total L_2 error/loss of 0.00100.

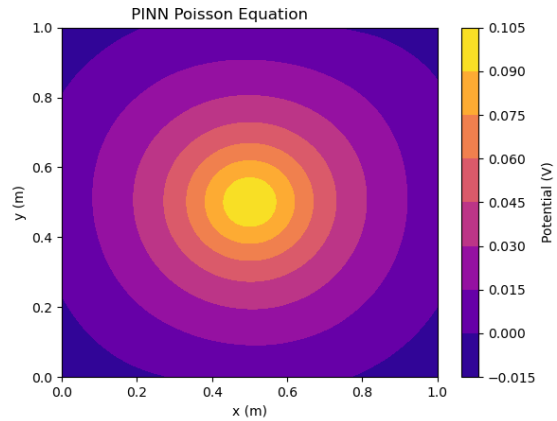

FIG. 2. Contour plot of the PINN solution to Poisson's equation.

For Laplace's equation, there are more solid error metrics; and so, though it may be a particular case of Poisson's, more time will be spent discussing the solutions found. First, the SOR solution to Laplace's equation, shown in FIG. 3, was attained after 292 iterations in around 12 seconds. The algorithm stops when it becomes close enough to the known analytical center value; in this instance, where all boundaries are set at 0 Volts but the top, which is set at 100 Volts, the center of the area should have a value of 25 Volts.[2] When it reaches 24.99 Volts, the algorithm stops.

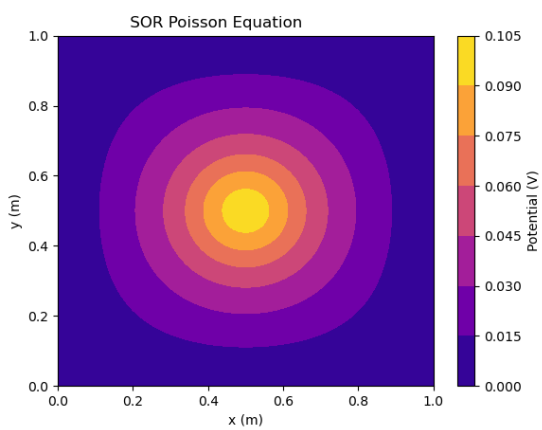

FIG. 1. Contour plot of the SOR method solution to Poisson's equation.

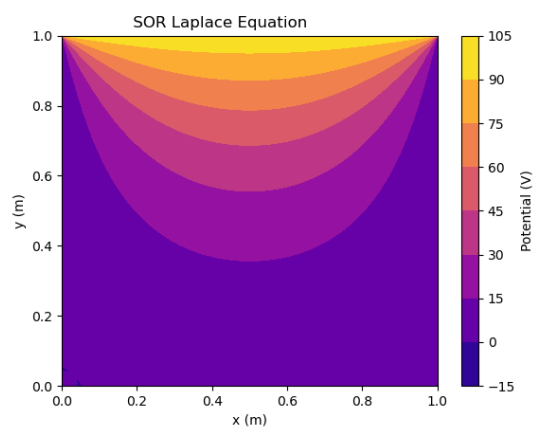

FIG. 3. Contour plot of the SOR solution to Laplace's equation.

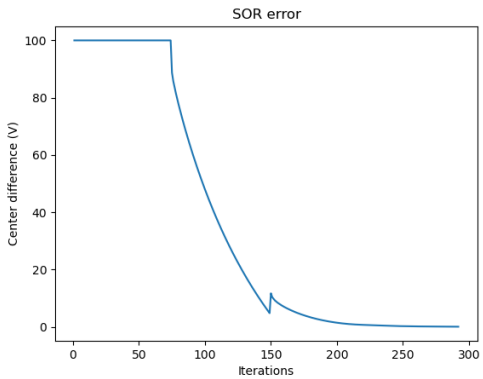

FIG. 4. Error in SOR Laplace solution during iteration.

The PINN solution in FIG. 5 was attained after 25000 epochs in approximately 5 minutes, and had an L_2 loss of 363. FIG 6 shows a plot of loss as the epochs progress, where it seems to converge near the end.

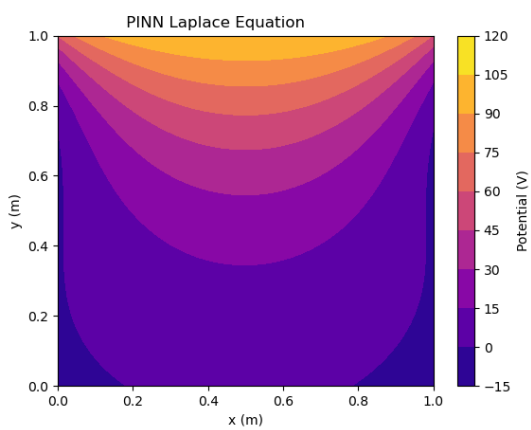

FIG. 5. PINN solution for Laplace's equation.

The center of the SOR solution differs from the true value by around 0.04 %, while the center of the PINN solution differs by 5 %.

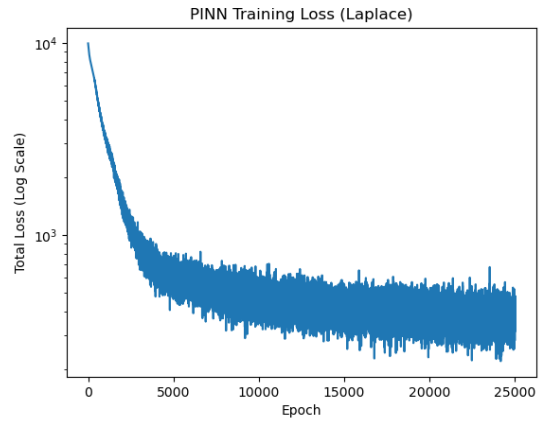

FIG. 6. Loss in the Laplace PINN solution as training progresses.

The two methods generally seem to agree, though in both cases the PINN solution struggles to properly fit the boundaries, where most of the error is. Neither PINN solution corresponds exactly on the boundaries. Both get the 'general shape' of the potential within reason, but fail to properly force the boundaries to stay constant.

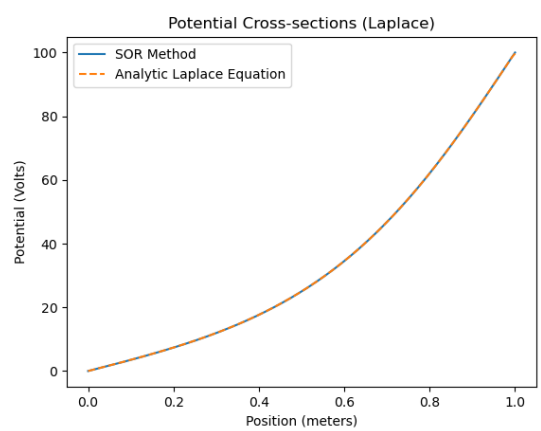

FIG. 7. Cross-sectional comparison of the SOR and Analytical solutions to Laplace's equation.

FIG. 8 shows a plot of the analytical solution, which is fairly similar to both numerical solutions, though is certainly closer to SOR.

This is found through a seperation of variables, though it is an infinite series, and so presented here is only the solution accurate to 100 terms. Still, this is a verified approximation.

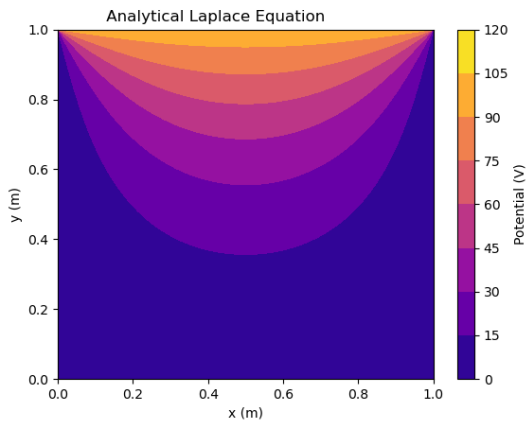

FIG. 8. Analytical solution of Laplace's equation, calculated to 100 terms.

4 Analysis

A numerical solution, such as SOR, is a very general tool, in that it seems to preform fairly well in most circumstances. Of course, as the number of points grows, the number of calculations required grows as well rather quickly, and so for very large numbers of discrete points, the numerical solution may not be the best. Especially here, though, while it can be slow when used repetitively, it is very accurate.

The PINN solution is slower to train, but is much faster for successive uses; in that the model only needs to calculate a value for each point once, while the numerical solution has to do the entire thing each time. The PINN solution is also much less accurate (or requires a much grander algorithm for the same level of accuracy). As such, I would say that the numerical solution is the better choice in general, though the PINN may do well in a problem where you need to calculate several times in succession (as having this sort of 'ready-made' solution will be rather handy).

A significant disadvantage present in the PINN model is that it is a good deal more complex, and so it requires more time to configure. Especially in that the actual structure may need to be experimented with itself to solve a particular problem.

Of note as well is the fact that the PINN produces a smooth solution, due to its use of automatic differentiation.

One last note: the main issue with the PINN solutions was the boundaries, in that they didn't follow the boundary conditions too well. Most of the present loss was due to this, and so it seems reasonable to say that the reason for this is the boundary part of the function getting stuck in a local minimum.

The biggest surprise this project held for me was how complex actually setting up the PINN training was. It takes a bit of time to turn the knobs about and configure it properly for the problem. Finding the middle point between over and under-determination (which is, of course, determination itself - precisely the goal!).

5 Conclusions

In essence, the results here seem to indicate that, in this particular circumstance, the SOR method is a better choice than a PINN model. In particular, the PINN model is rather inaccurate around the boundaries of the square area. This being said, a more sophisticated PINN will likely proform much better.

Intent learning experience! Of course, solving PDEs is rather important aspect to many problems, and something I had not done very much previous to this, especially with regard to numerical solutions and such. I went directly through implementing Jacobi's method and modifying it into SOR,

which was informative to see. Also, I had not previously used PyTorch, or any packages of that sort, and so it was interesting as well to manipulate the PINN model and attempt to find values for all of the various components of it that worked well with the problem.

It is a very lovely feeling to have optimized something so that it fetches a solution quickly. Joy of seeing a system you set up work properly!

An obvious extension would be to transport this to three dimensions, but perhaps more interestingly, the PINN solutions could certainly be improved. There are several methods that could likely be used to get past the apparent local minimum in the Laplace PINN function; an implementation of some manner of escaping local minima would be a natural next step.

(This is quite late because this is exactly what I have been trying to do, but at this point I think it prudent to leave it be and conclude for now. I had played around with getting MCMC, or something of the like, to work in the loss function, but have been thus far unsuccessful. It is sad, it could be so much better! C'est la vie, I suppose.)

6 References

[1] (https://people.eecs.berkeley.edu/ demmel/cs267/lecture24/lecture24.html) (1996)

[2] (https://www.youtube.com/watch?v=ws3LIfWxfcMt=714s) (2014)